Documento de Arquitectura

INGENIERIA DE SOFTWARE 2

Presentado por:
Juan Camilo Henao Villegas
Juan Pablo Collazos Samboni
Santiago Majé Bonilla
Daniel Alexander Chaguendo

Profesor:
ROBERTO ENCARNACION MOSQUERA

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones
Ingeniería en Sistemas
Popayán, Marzo del 2026

CONTENIDO

1. Historias de usuario y criterios de aceptación 1
2. Prototipos de interfaz de usuario y thinking aloud 2
3. Planificación de tareas en Jlra 3
4. Escenario de calidad modificabilidad 4
5. Arquitectura y diseño de software 5
6. Patrones de diseño implementados 7

Índice de ilustraciones

No se encuentran elementos de tabla de ilustraciones.

Índice de tablas

Tabla 1. Escenario 1: Recuperación ante falla del servidor durante horario de agendamiento. 6
Tabla 2. Escenario 2 :Manejo de carga concurrente. 7
Tabla 3. Escenario 3: Intento de acceso no autorizado a historia clínica 8
Tabla 4. Escenario 4: Ataque de fuerza bruta al sistema de autenticación 9

Introducción

Este documento presenta la arquitectura de software del sistema de reserva de citas médicas desarrollado como parte del curso de Ingeniería de Software II. Su propósito es describir la estructura general de la aplicación, los componentes que la conforman y las decisiones de diseño adoptadas para garantizar atributos de calidad como la modificabilidad, mantenibilidad y escalabilidad.

El sistema se implementa bajo una arquitectura monolítica en tres capas, haciendo uso del micro-patrón MVC, así como de principios SOLID y patrones de diseño GoF. A lo largo del documento se detallan las historias de usuario implementadas, los prototipos de interfaz, los escenarios de calidad y los diferentes niveles de la arquitectura utilizando el modelo C4.

Historias de usuario y criterios de aceptación

		Enunciado de la historia				Criterios de aceptación			
Identificador (ID) de la épica	Identificador (ID) de la historia	Rol	Característica / Funcionalidad (Quiero)	Razón / Resultado (Para)	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
	HU-1.1.	Yo como administrador	Crear usuarios del sistema (médicos y terapistas)	Permitir al personal autorizado acceder al sistema con un rol específico	1	Creación exitosa	El administrador diligencia login único, contraseña, nombre completo, rol y estado	Cuando presiono en guardar	El sistema crea el usuario, almacena contraseña cifrada y muestra mensaje de éxito
					2	Login duplicado	Se ingresa un login ya existente	Cuando presiono en guardar	El sistema muestra error "Login ya existe"
					3	Asociación profesional	El rol es Médico/Terapista	Cuando presiono en guardar	El sistema obliga a seleccionar el profesional correspondiente
	HU-1.2.	Yo como administrador	Quiero editar la información de un usuario	Para mantener actualizados los datos del sistema	1	Edición exitosa	El administrador accede al LISTADO	Cuando presiono en	El sistema actualiza la información correctamente y muestra mensaje "Datos actualizados exitosamente".
					3	Cambio de estado	Administrador cambia estado a ACTIVO/ INACTIVO	Cuando presiono en guardar	Usuario puede/ no puede iniciar sesión
	HU-1.3.	Yo como administrador	Quiero consultar usuarios	Para verificar su estado y rol	1	Listado completo	Administrador accede al módulo	Abrir pantalla	Sistema muestra listado con login, nombre, rol y estado
					2	Filtro por rol o estado	Administrador selecciona filtro	Aplicar filtro	Sistema muestra solo usuarios del rol seleccionado
	HU-1.4.	Yo como administrador	Quiero desactivar un usuario	Para impedir su acceso sin borrar historial	1	Desactivación exitosa	Usuario activo seleccionado	Presiona en desactivar	Estado cambia a INACTIVO
HE-01					2	Intento de login	Usuario inactivo intenta iniciar	Login	Sistema bloquea acceso

Imagen 1. Historias de usuario y criterios de aceptación de la épica 1.

		Enunciado de la historia				Criterios de aceptación			
Identificador (ID) de la épica	Identificador (ID) de la historia	Rol	Característica / Funcionalidad (Quiero)	Razón / Resultado (Para)	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
	HU-5.1	Como Usuario autenticado	Quiero iniciar el asistente de agendamiento	Para solicitar una cita médica de forma autónoma	1	Acceso autenticado	Usuario con sesión activa	Clic en AGENDAR CITA	Sistema muestra asistente de agendamiento
					2	Usuario no autenticado	Usuario sin sesión	Intenta acceder	Sistema redirige a INICIO DE SESIÓN
					3	Control por rol	Usuario con rol no permitido	Intenta acceder	Sistema bloquea acceso
	HU-5.2	Como Usuario autenticado	Quiero seleccionar especialidad o profesional	Para recibir atención adecuada	1	Mostrar especialidades	Usuario accede al recorrido	Pantalla carga	Sistema muestra: Terapia neural, Quiropraxia, Fisioterapia
					2	Mostrar profesionales activos	Usuario selecciona especialidad	Continuar	Sistema muestra solo médicos/terapistas activos
					3	Profesional inactivo	Profesional está inactivo	Consulta disponibilidad	Sistema no permite seleccionarlo
	HU-5.3	Como Usuario autenticado	Quiero que el sistema me sugiera fechas y horas disponibles	Para agendar la cita más cercana posible	1	Sugerencia automática	Profesional seleccionado	Consultar disponibilidad	Sistema calcula fecha/hora más cercana
					2	Evitar duplicidad	Otro usuario intenta reservar	Confirmar cita	Sistema impide duplicación (control de
					3	Festivo	Fecha seleccionada es	Confirmar	Sistema bloquea selección
	HU-5.4	Como Usuario autenticado	Quiero confirmar la cita seleccionada	Para reservar el horario elegido	1	Confirmación exitosa	Horario disponible	Clic en CONFIRMAR	Sistema guarda cita con fecha, hora, profesional y
					2	Registro auditoría	Cita creada	Clic en CONFIRMAR	Sistema registra quién creó la cita y cuándo
HE-05					3	Validación de datos	Información incompleta	Clic en CONFIRMAR	Sistema bloquea y muestra error

Imagen 2. Historias de usuario y criterios de aceptación de la épica 5.

1. Prototipos de interfaz de usuario y thinking aloud

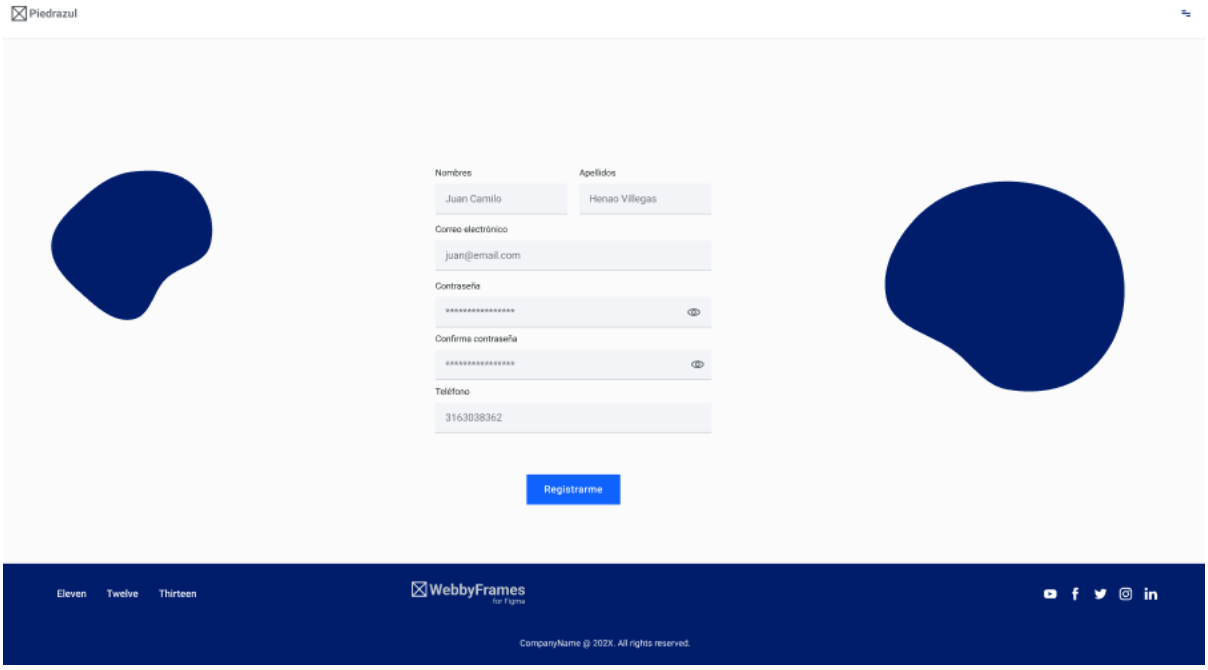


Imagen 3. Prototipo de interfaz de usuario.

2.1 Thinking Aloud

“Bueno, estoy entrando a la aplicación… lo primero que noto es que la interfaz se ve bien, los colores están agradables, no chocan ni nada, todo se ve limpio. Aunque… siento que se ve más como una página web que como un programa de escritorio. No sé, como que le falta un poco más de personalidad o elementos más propios de una app de escritorio.

Voy a navegar un poco… entro al dashboard del administrador… ok, aquí hay varias opciones. Voy a ‘gestionar usuarios’. Mmm… algo que noto es que no es tan claro en qué pestaña estoy. O sea, sí cambié de vista, pero no hay algo que resalte o me indique claramente que estoy en esta sección. Me tocaría fijarme bien en el contenido para saberlo, y eso no debería pasar.

Sigo mirando… los colores están bien, como dije, pero tal vez se podría agregar otro color complementario para dar más contraste o jerarquía visual, especialmente para resaltar cosas importantes como botones o secciones activas.

Ahora, viendo esta pantalla… siento que hay muchos espacios vacíos. Como que hay partes donde podría haber más información o mejor distribución de los elementos. Se ve un poco desaprovechado el espacio, y eso hace que la interfaz se sienta menos dinámica.

En general, la app se entiende, no es confusa, pero sí le falta ese toque más ‘aplicación de escritorio’, mejorar la indicación de navegación (en qué sección estoy), y optimizar el uso del espacio.”

Planificación de tareas en JIra

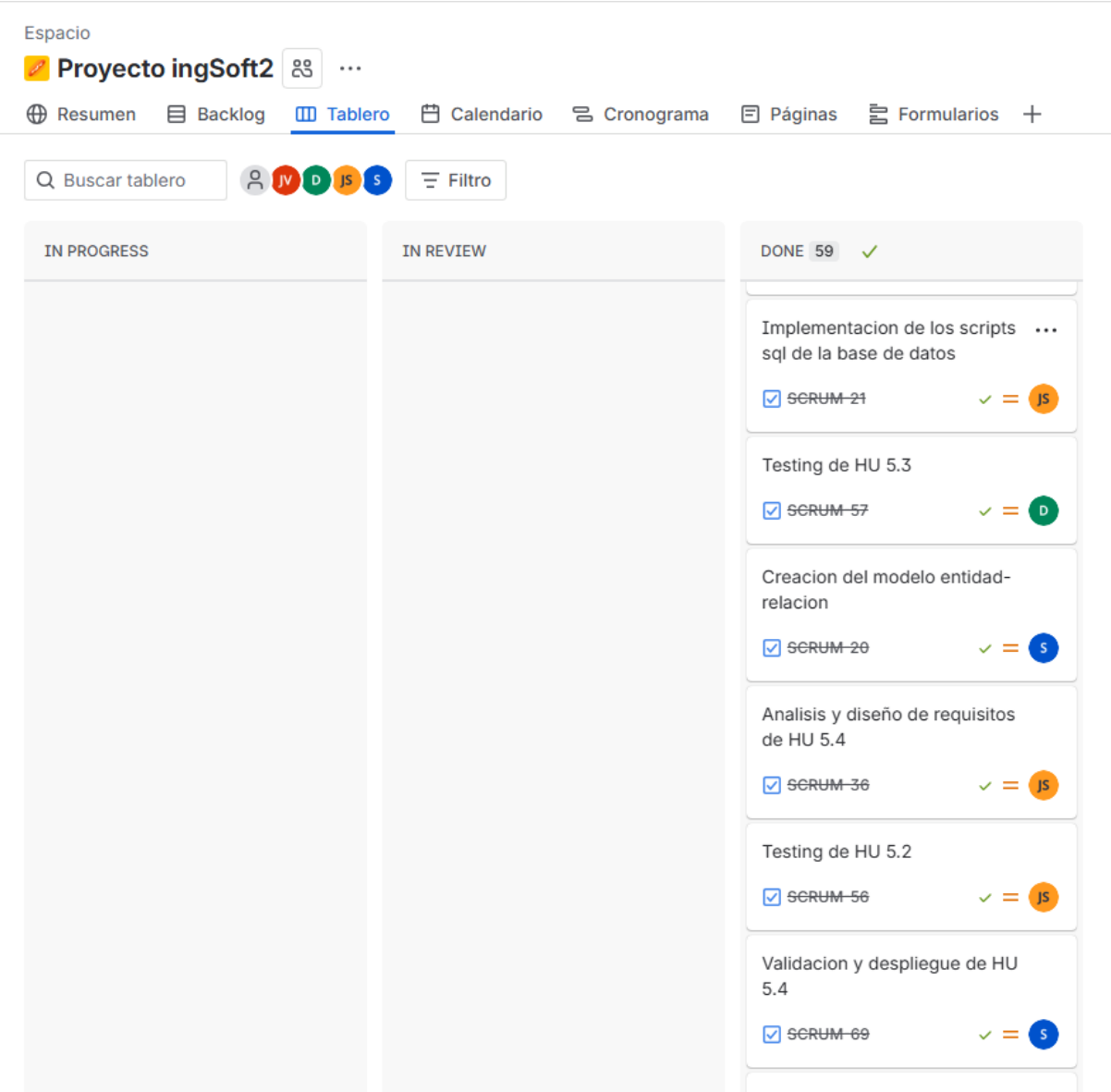


Imagen 4. Pantallazo de las tareas en Jira

2. Escenario de calidad modificabilidad

Porción del escenario	Descripción

Fuente	Equipo de desarrollo / Arquitecto de software
Estímulo	Se requiere refactorizar la aplicación monolítica para adoptar una arquitectura de microservicios basada en eventos, con el fin de mejorar la escalabilidad y mantenibilidad
Artefacto	Arquitectura del sistema (capas: presentación, negocio y persistencia) y módulos de dominio (citas, usuarios, configuración)
Entorno	Sistema en operación, con necesidad de evolución tecnológica sin interrupción del servicio
Respuesta	El sistema permite desacoplar progresivamente los módulos del monolito en servicios independientes, integrados mediante eventos (por ejemplo, publicación y suscripción), sin afectar el funcionamiento actual
Medida de respuesta	- Refactorización incremental por módulos (sin reescritura completa) ▪ Bajo acoplamiento entre componentes (facilitando su extracción como servicios) ▪ Introducción de un mecanismo de eventos sin impactar la lógica existente ▪ Disponibilidad del sistema ≥ 99% durante la migración ▪ Tiempo de adaptación por módulo ≤ 1 semana **Resultado esperado** | El sistema evoluciona hacia una arquitectura de microservicios orientada a eventos de manera gradual, reduciendo riesgos, manteniendo la continuidad del servicio y facilitando futuras extensiones

Justificación

Este escenario permite evaluar la capacidad del sistema para evolucionar desde una arquitectura monolítica hacia un enfoque de microservicios orientado a eventos, sin necesidad de rediseñar completamente la aplicación. Gracias a la separación por capas, el uso del patrón MVC, y la aplicación de principios SOLID, los módulos del sistema pueden desacoplarse progresivamente.

La introducción de comunicación basada en eventos (por ejemplo, mediante un broker de mensajería) permite reducir dependencias directas entre componentes, facilitando la escalabilidad y la incorporación de nuevos servicios en el futuro. Esto

asegura que el sistema no solo sea modificable a nivel funcional, sino también a nivel arquitectónico.

3. Arquitectura y diseño de software

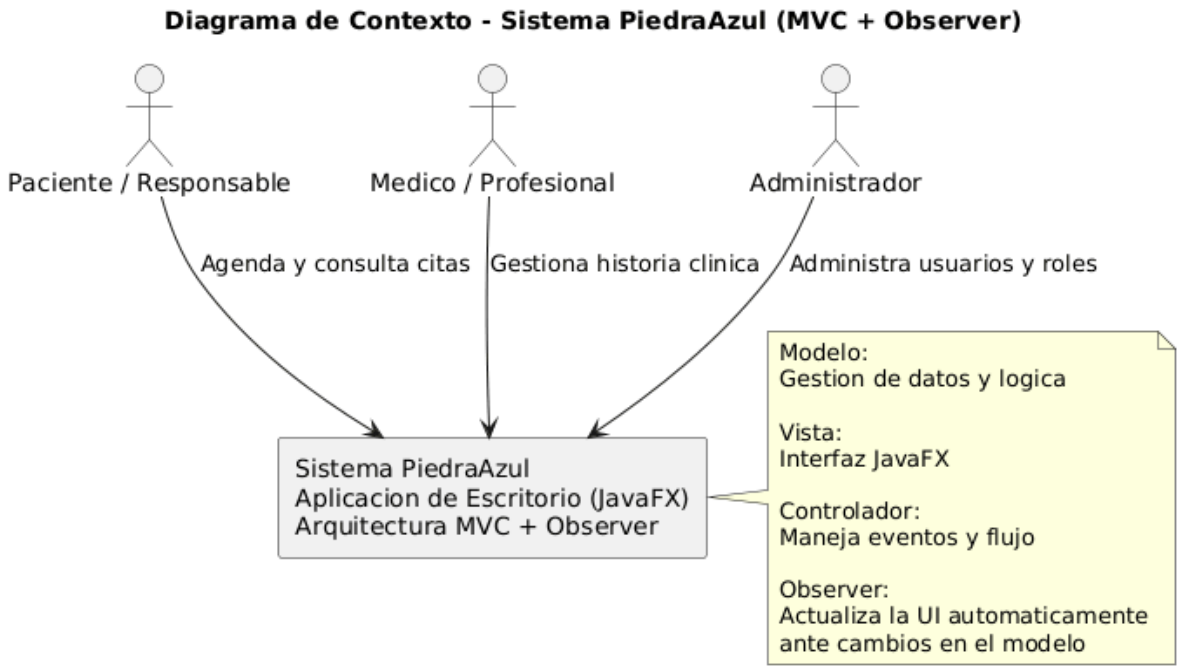


Imagen 5. Diagrama de contexto

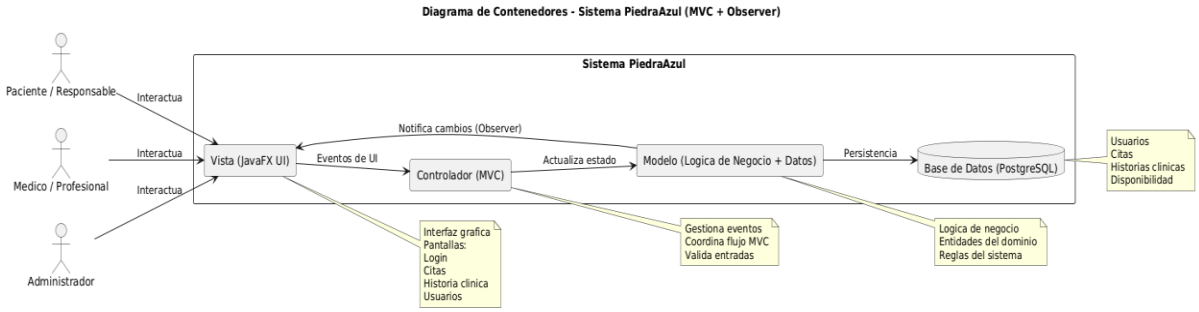


Imagen 6. Diagrama de contenedores

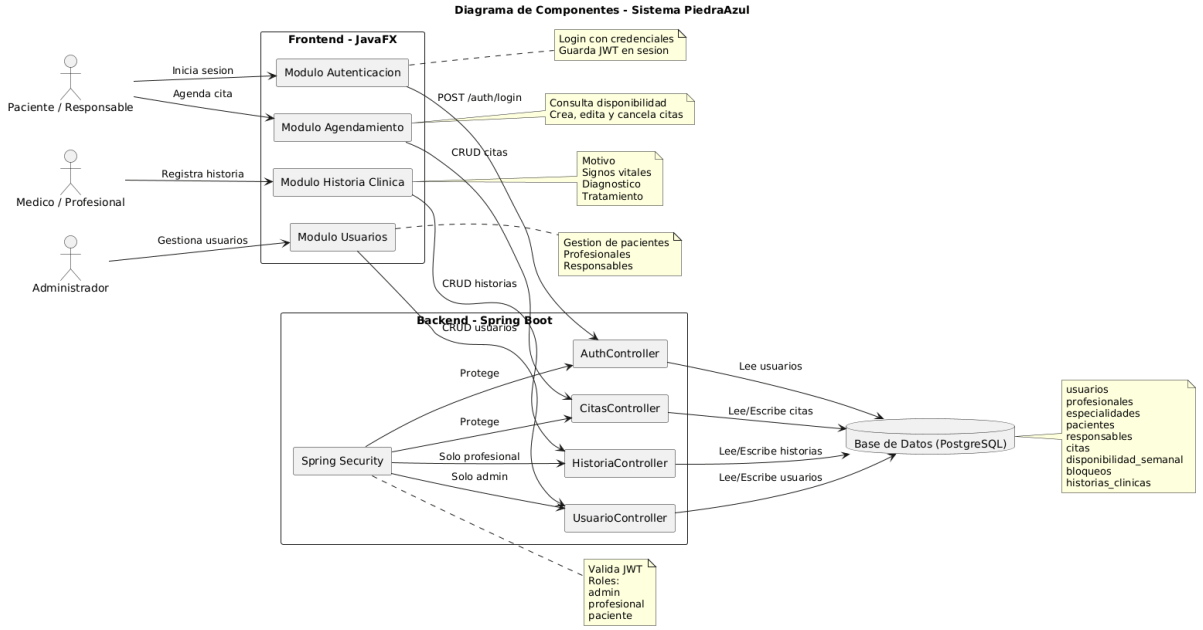


Imagen 7. Diagrama de componentes.

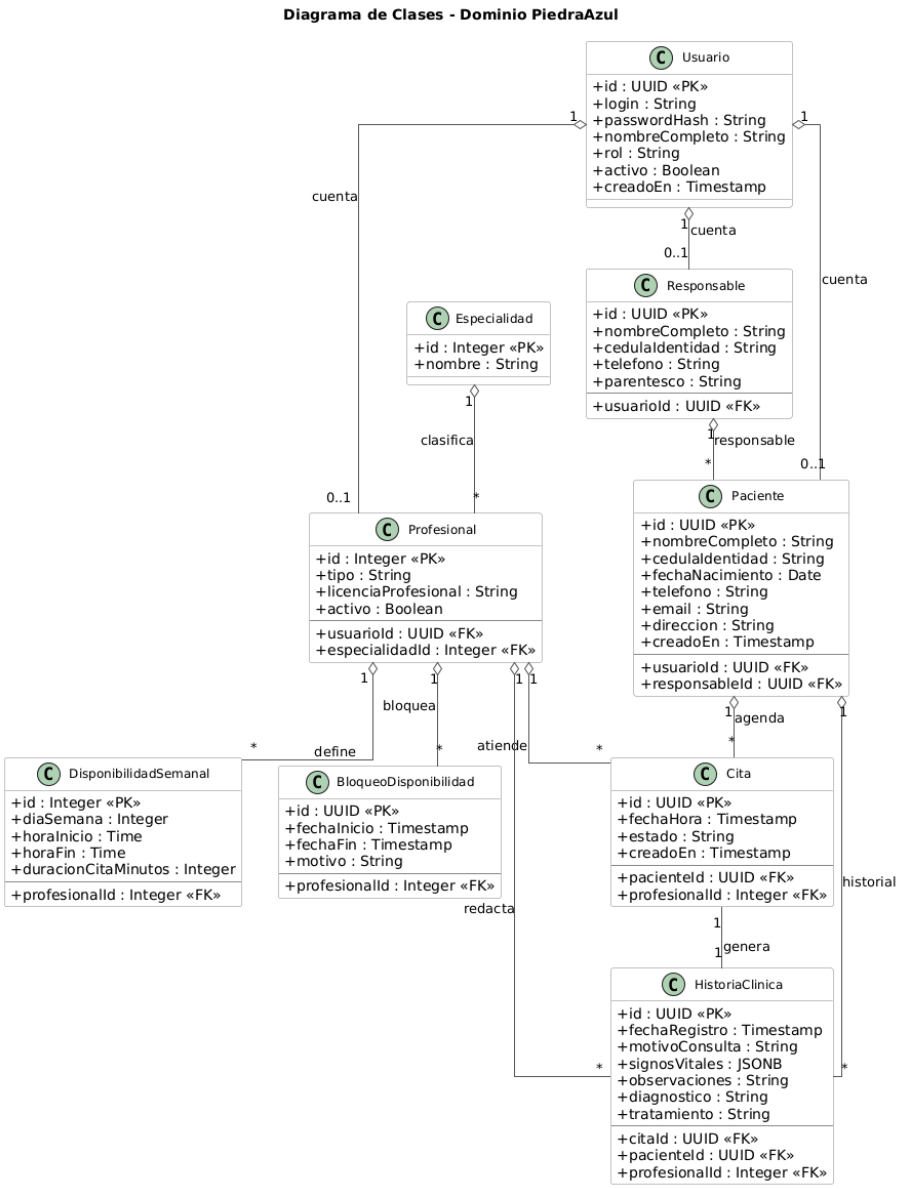


Imagen 8. Diagrama de clases.

4. Patrones de diseño implementados

Modelo-Vista-Controlador (MVC)

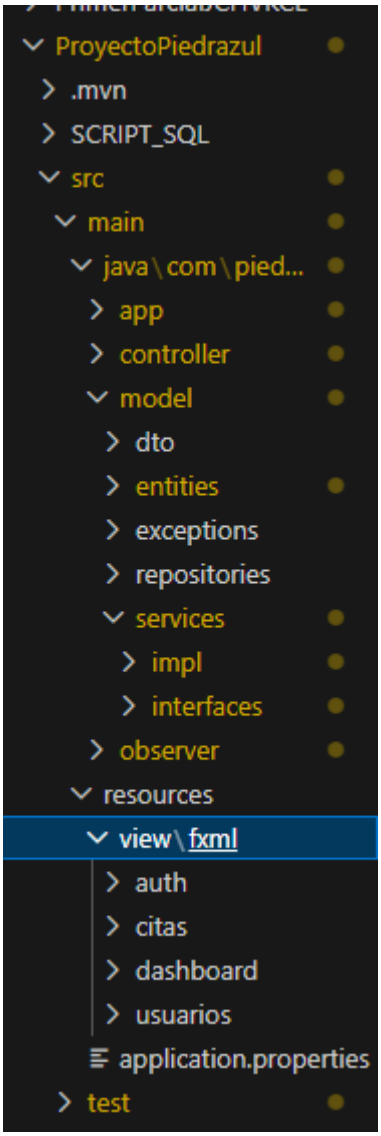
Tipo: Arquitectónico / Micro-patrón

Descripción:
El patrón MVC divide la aplicación en tres componentes principales:

- **Modelo:** Representa los datos y la lógica del negocio.
- **Vista:** Encargada de la interfaz gráfica de usuario.

- **Controlador:** Gestiona la interacción del usuario y coordina la comunicación entre modelo y vista.

Contexto de uso:
Se implementó en la aplicación Java Desktop para estructurar la interfaz gráfica y separar la lógica de negocio de la presentación. Esto permite modificar la interfaz sin afectar la lógica del sistema, y viceversa, mejorando la mantenibilidad y la modificabilidad.



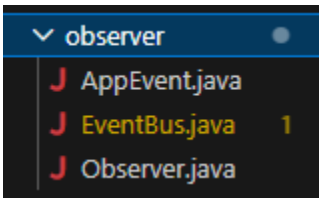
Observer

Tipo: Comportamiento

Descripción:
El patrón Observer establece una relación de dependencia uno a muchos entre

objetos, de manera que cuando un objeto cambia su estado, notifica automáticamente a sus observadores.

Contexto de uso:
Se utilizó para gestionar la actualización de componentes del sistema ante cambios en el estado, por ejemplo, cuando se crea o modifica una cita, permitiendo que la interfaz u otros componentes reaccionen automáticamente sin generar acoplamientos directos.



Conclusión

La implementación de los patrones MVC y Observer permite una adecuada separación de responsabilidades y comunicación desacoplada entre componentes. Esto contribuye directamente al atributo de calidad de modificabilidad, facilitando la evolución del sistema y la incorporación de nuevos requerimientos sin afectar significativamente la estructura existente.

Links

CamiloHenaoV/ProyectoPiedrazul

https://www.figma.com/design/0TBfCOD3tlrNeGYASVDkCB/Wireframe-piedrazul?node-id=1140-6414&t=d8jU9UvfTXkBFKFa-1

https://docs.google.com/spreadsheets/d/1mAnrQiilUVUWu-FfYbVVNTRbaDRx6hNZ3DpW-x1-V3Y/edit?usp=sharing

https://unicauca-team-kihnzy43.atlassian.net/jira/software/projects/SCRUM/summary?atlOrigin=eyJpIjoiMTM3NDRiMzZiN2M2NDk3OTlkNjM0MDZmOGE4ZGZhZDMiLCJwIjoiaiJ9

-- Fin del documento.

Referencias

Anexos

1.https://docs.google.com/spreadsheets/d/1mAnrQiilUVUWu-FfYbVVNTRbaDRx6hNZ3DpW-x1-V3Y/edit?usp=sharing

2.https://www.figma.com/design/0TBfCOD3tlrNeGYASVDkCB/Wireframe-piedrazul?node-id=1140-6414&t=gD8UkpV5iaTShRHS-1